ROI for AI: Midterm Report

Yohannes Belay Woldeyes
Introduction to AI and Machine Learning, MCSBT, IE University

1. The idea

This is what I posted in the "ROI for AI" discussion:

> *"Enterprise sales engineers lose substantial time manually searching through fragmented technical documentation to answer repetitive enterprise RFPs, creating a bottleneck that limits deal flow.*
>
> *An internal GenAI engine indexed on past winning proposals can automatically draft verified answers to routine questions, freeing engineers to focus solely on expert review while enabling the business to pursue far more bids with existing staff."*

The return comes from two places: engineers spend less time looking for material they have already written, and the same team can respond to more bids. For the midterm I built the search part of this idea. You type in a requirement and get back the 50 closest matching notices, ranked by Word2Vec similarity. The notebook does not draft answers, so this report only makes claims about search.

2. How I built it

Real proposals are private, so I needed a public stand-in. I used EuroTender, a set of EU public procurement notices from Tenders Electronic Daily, published under CC-BY 4.0 at huggingface.co/datasets/sinceaihq/eurotender-cpv. The full dataset has 824,201 notices and 18 columns. My subset keeps the 15,952 English notices from October 2025 to September 2026 and the 6 columns I needed, and it is submitted as rfp_ted_notices.csv. It covers all kinds of procurement, not only IT: 8,618 services, 5,368 supplies and 1,966 works notices. The file has the same shape as the Crunchbase file, with notice_id in place of crunchbase_uuid, title in place of name and description in place of short_description.

The PySpark notebook follows the professor's template and ran on Spark 4.0.4 in Colab. I loaded the CSV into a Spark DataFrame and removed repeated titles, because the result is a list of titles and each should appear once. That left 15,540 notices. I tokenized the descriptions with RegexTokenizer and removed stopwords with StopWordsRemover, which took the text from 1,309,699 tokens to 833,258. I then trained Word2Vec on the output with 100-dimensional vectors and a minimum word count of 5. It uses Skip-Gram, so each word is used to predict the words around it. Each notice and each search query becomes the average of its word vectors, and cosine similarity ranks all 15,540 notices against the query.

I changed the template in a few places. The text in this file writes quotes as two quote characters, so the CSV reader needed the escape option. I searched every notice instead of using the 30,000-row limit, because the dataset is small enough. The change that mattered most was the number of training passes. With the default of one pass, only 4 of the top 50 results for the cybersecurity query were relevant, which is a sign of underfitting, so I trained for 10 passes with a fixed seed. Table 1 shows the effect, with the 1-pass numbers coming from a separate run with maxIter set to 1. After 10 passes the closest words to "security" were cybersecurity, soc, incidents, privacy and threat.

Table 1. Relevant notices in the top 50, using the keyword check in section 3

Query	1 pass	10 passes
cybersecurity threat monitoring vulnerability assessment	4	31
cloud migration database infrastructure management	32	39
data protection privacy compliance	10	13

3. Results

I tested three queries, and the notebook shows the full top 50 titles for each. The data has no labels saying which notice answers which query, so I used a simple check: a notice counts as relevant if its title or description contains one of the topic terms I chose for that query. Precision at 50 is the share of the top 50 that are relevant. The base rate is the share of all notices that are relevant, which is what picking at random would give on average. Lift is precision divided by the base rate. Because this only checks for matching words, I also read the titles.

Table 2. Evaluation of the three queries after 10 passes

Query	Relevant in top 50	Precision at 50	Base rate	Lift	On topic when I read the titles
Cybersecurity	31	0.62	1.4%	45.2	About 31
Cloud and database	39	0.78	3.3%	23.8	About 21
Data protection	13	0.26	0.6%	43.9	5 to 9

All three queries do far better than chance, but the results are not equally good. Cybersecurity was the strongest, and its top 8 results are all about cybersecurity. The keyword count is too high for the cloud query, because it also counts general IT systems such as CRM, HR and print services. Data protection was the weakest: only 0.6% of notices mention the topic, so most results are nearby security and compliance notices.

The method also has clear limits. Averaging word vectors loses the meaning of a word in context, so “security” also brings in guarding and travel risk notices, and the data protection query returned “Vulnerability Assessments to International Protection Applicants”. A high similarity score does not guarantee a relevant notice. These notices are also written by buyers, so the search finds similar requirements, not past answers.

4. What this means for the ROI

The search works as a shortlist for an engineer to review, not as a final answer. About 31 of the 50 titles were on topic for the strongest query and only 5 to 9 for the weakest, so someone still has to check each result. To reuse past answers, the same method would have to run on a company’s own proposals.

This midterm did not measure time saved or extra bids won, so I am not claiming a financial figure. To work out the return, a company could run a pilot on its own archive and measure the inputs to this calculation:

Yearly benefit = (search hours saved per RFP × RFPs per year × engineer’s hourly cost) + (extra bids per year × win rate × margin per win)

ROI = (yearly benefit − yearly cost of the system) / yearly cost of the system